What's new in .NET MAUI for .NET 9

Article • 11/08/2024

The focus of .NET Multi-platform App UI (.NET MAUI) in .NET 9 is to improve product quality. This includes expanding test coverage, end to end scenario testing, and bug fixing. For more information about the product quality improvements in .NET MAUI 9, see the following release notes:

- .NET MAUI 9 RC2
- .NET MAUI 9 RC1
- .NET MAUI 9 Preview 7
- .NET MAUI 9 Preview 6
- .NET MAUI 9 Preview 5
- .NET MAUI 9 Preview 4
- .NET MAUI 9 Preview 3
- .NET MAUI 9 Preview 2
- .NET MAUI 9 Preview 1

> **Important**
>
> Due to working with external dependencies, such as Xcode or Android SDK Tools, the .NET MAUI support policy differs from the .NET and .NET Core support policy. For more information, see .NET MAUI support policy.

Compatibility with Xcode 16, which includes SDK support for iOS 18, iPadOS 18, tvOS 18, and macOS 15, is required when building with .NET MAUI 9. Xcode 16 requires a Mac running macOS 14.5 or later.

In .NET 9, .NET MAUI ships as a .NET workload and multiple NuGet packages. The advantage of this approach is that it enables you to easily pin your projects to specific versions, while also enabling you to easily preview unreleased or experimental builds. When you create a new .NET MAUI project the required NuGet packages are automatically added to the project.

Minimum deployment targets

.NET MAUI 9 requires minimum deployment targets of iOS 12.2, and Mac Catalyst 15.0 (macOS 12.0). Android and Windows minimum deployment targets remain the same. For more information, see Supported platforms for .NET MAUI apps.

New controls

.NET MAUI 9 includes two new controls.

HybridWebView

HybridWebView enables hosting arbitrary HTML/JS/CSS content in a web view, and enables communication between the code in the web view (JavaScript) and the code that hosts the web view (C#/.NET). For example, if you have an existing React JS app, you could host it in a cross-platform .NET MAUI native app, and build the back-end of the app using C# and .NET.

To build a .NET MAUI app with HybridWebView you need:

- The web content of the app, which consists of static HTML, JavaScript, CSS, images, and other files.
- A HybridWebView control as part of the app's UI. This can be achieved by referencing it in the app's XAML.
- Code in the web content, and in C#/.NET, that uses the HybridWebView APIs to send messages between the two components.

The entire app, including the web content, is packaged and runs locally on a device, and can be published to applicable app stores. The web content is hosted within a native web view control and runs within the context of the app. Any part of the app can access external web services, but isn't required to.

For more information, see HybridWebView.

Titlebar for Windows

The TitleBar control provides the ability to add a custom title bar to your app on Windows:

A TitleBar can be set as the value of the Window.TitleBar property on any TitleBar:

XAML

```
<Window.TitleBar>
    <TitleBar x:Name="TeamsTitleBar"
              Title="Hello World"
```

```
                Icon="appicon.png"
                HeightRequest="46">
        <TitleBar.Content>
            <SearchBar Placeholder="Search"
                       PlaceholderColor="White"
                       MaximumWidthRequest="300"
                       HorizontalOptions="Fill"
                       VerticalOptions="Center" />
        </TitleBar.Content>
    </TitleBar>
</Window.TitleBar>
```

An example of its use in C# is:

C#

```
Window window = new Window
{
    TitleBar = new TitleBar
    {
        Icon = "titlebar_icon.png"
        Title = "My App",
        Subtitle = "Demo"
        Content = new SearchBar { ... }
    }
};
```

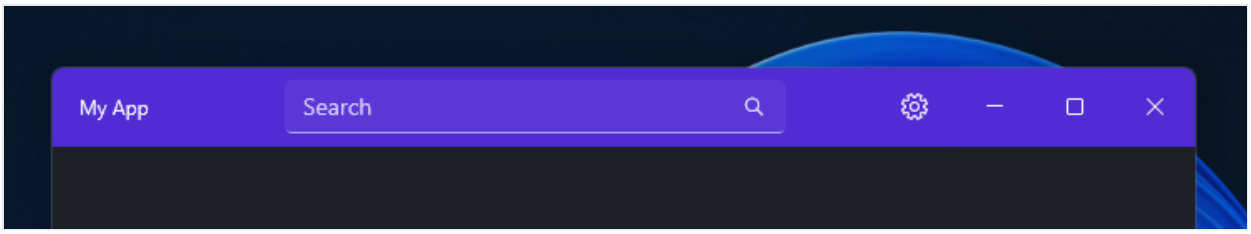
A TitleBar is highly customizable through its Content, LeadingContent, and TrailingContent properties:

XAML

```
<TitleBar Title="My App"
          BackgroundColor="#512BD4"
          HeightRequest="48">
    <TitleBar.Content>
        <SearchBar Placeholder="Search"
                   MaximumWidthRequest="300"
                   HorizontalOptions="Fill"
                   VerticalOptions="Center" />
    </TitleBar.Content>
    <TitleBar.TrailingContent>
        <ImageButton HeightRequest="36"
                     WidthRequest="36"
                     BorderWidth="0"
                     Background="Transparent">
            <ImageButton.Source>
                <FontImageSource Size="16"
                                 Glyph="&#xE713;"
                                 FontFamily="SegoeMDL2"/>
            </ImageButton.Source>
```

```
        </ImageButton>
    </TitleBar.TrailingContent>
</TitleBar>
```

The following screenshot shows the resulting appearance:



> [!NOTE]
> Mac Catalyst support for the `TitleBar` control will be added in a future release.

For more information, see TitleBar.

Control enhancements

.NET MAUI 9 includes control enhancements.

BackButtonBehavior OneWay binding mode

The binding mode for `IsVisible` and `IsEnabled` on a BackButtonBehavior in a Shell app is now `BindingMode.OneWay` instead of `BindingMode.OneTime`. This enables you to more easily control the behavior of the back button at runtime, with data bindings:

XAML

```
<ContentPage ...>
    <Shell.BackButtonBehavior>
        <BackButtonBehavior Command="{Binding BackCommand}"
                            IsVisible="{Binding IsBackButtonVisible}"
                            IconOverride="back.png" />
    </Shell.BackButtonBehavior>
    ...
</ContentPage>
```

BlazorWebView

On iOS and Mac Catalyst 18, .NET MAUI 9 changes the default behavior for hosting content in a BlazorWebView to `localhost`. The internal `0.0.0.1` address used to host content no longer works and results in the BlazorWebView not loading any content and rendering as an empty rectangle.

To opt into using the `0.0.0.1` address, add the following code to the `CreateMauiApp` method in *MauiProgram.cs*:

```
C#

// Set this switch to use the LEGACY behavior of always using 0.0.0.1
to host BlazorWebView
AppContext.SetSwitch("BlazorWebView.AppHostAddressAlways0000", true);
```

If you encounter hangs on Android with BlazorWebView you should enable an AppContext switch in the `CreateMauiApp` method in your `MauiProgram` class:

```
C#

AppContext.SetSwitch("BlazorWebView.AndroidFireAndForgetAsync",
true);
```

This switch enables BlazorWebView to fire and forget the async disposal that occurs, and as a result fixes the majority of the disposal deadlocks that occur on Android. For more information, see Fix disposal deadlocks on Android.

Buttons on iOS

Button controls on iOS now respect spacing, padding, border width, and margins more accurately than in previous releases. A large image in a Button will now be resized to the maximum size, taking into account the spacing, padding, border width, and margins. However, if a Button contains text and an image it might not be possible to fit all the content inside the button, and so you should size your image manually to achieve your desired layout.

CollectionView and CarouselView

.NET MAUI 9 includes two optional new handlers on iOS and Mac Catalyst that bring performance and stability improvements to `CollectionView` and `CarouselView`. These handlers are based on `UICollectionView` APIs.

To opt into using these handlers, add the following code to your `MauiProgram` class:

C#

```
#if IOS || MACCATALYST
builder.ConfigureMauiHandlers(handlers =>
{
    handlers.AddHandler<Microsoft.Maui.Controls.CollectionView,
Microsoft.Maui.Controls.Handlers.Items2.CollectionViewHandler2>();
    handlers.AddHandler<Microsoft.Maui.Controls.CarouselView,
Microsoft.Maui.Controls.Handlers.Items2.CarouselViewHandler2>();
});
#endif
```

ContentPage

In .NET MAUI 9, the HideSoftInputOnTapped property is also supported on Mac Catalyst, as well and Android and iOS.

Soft keyboard input support

.NET MAUI 9 adds new soft keyboard input support for `Password`, `Date`, and `Time`. These can be enabled on Editor and Entry controls:

XAML

```
<Entry Keyboard="Date" />
```

Text alignment

The TextAlignment enumeration adds a `Justify` member that can be used to align text in text controls. For example, you can horizontally align text in a Label with `HorizontalTextAlignment.Justify`:

XAML

```
<Label Text="Lorem ipsum dolor sit amet, consectetur adipiscing elit.
In facilisis nulla eu felis fringilla vulputate."
       HorizontalTextAlignment="Justify"/>
```

TimePicker

TimePicker gains a TimeSelected event, which is raised when the selected time changes. The TimeChangedEventArgs object that accompanies the `TimeSelected` event has

`NewTime` and `OldTime` properties, which specify the new and old time, respectively.

WebView

WebView adds a `ProcessTerminated` event that's raised when a WebView process ends unexpectedly. The `WebViewProcessTerminatedEventArgs` object that accompanies this event defines platform-specific properties that indicate why the process failed.

Compiled bindings in code

Bindings written in code typically use string paths that are resolved at runtime with reflection, and the overhead of doing this varies from platform to platform. .NET MAUI 9 introduces an additional SetBinding extension method that defines bindings using a `Func` argument instead of a string path:

C#

```
// in .NET 8
MyLabel.SetBinding(Label.TextProperty, "Text");

// in .NET 9
MyLabel.SetBinding(Label.TextProperty, static (Entry entry) =>
entry.Text);
```

This compiled binding approach provides the following benefits:

- Improved data binding performance by resolving binding expressions at compile-time rather than runtime.
- A better developer troubleshooting experience because invalid bindings are reported as build errors.
- Intellisense while editing.

Not all methods can be used to define a compiled binding. The expression must be a simple property access expression. The following examples show valid and invalid binding expressions:

C#

```
// Valid: Property access
static (PersonViewModel vm) => vm.Name;
static (PersonViewModel vm) => vm.Address?.Street;

// Valid: Array and indexer access
static (PersonViewModel vm) => vm.PhoneNumbers[0];
```

```
static (PersonViewModel vm) => vm.Config["Font"];

// Valid: Casts
static (Label label) => (label.BindingContext as
PersonViewModel).Name;
static (Label label) => ((PersonViewModel)label.BindingContext).Name;

// Invalid: Method calls
static (PersonViewModel vm) => vm.GetAddress();
static (PersonViewModel vm) => vm.Address?.ToString();

// Invalid: Complex expressions
static (PersonViewModel vm) => vm.Address?.Street + " " +
vm.Address?.City;
static (PersonViewModel vm) => $"Name: {vm.Name}";
```

In addition, .NET MAUI 9 adds a BindingBase.Create method that sets the binding directly on the object with a `Func`, and returns the binding object instance:

C#

```
// in .NET 8
myEntry.SetBinding(Entry.TextProperty, new MultiBinding
{
    Bindings = new Collection<BindingBase>
    {
        new Binding(nameof(Entry.FontFamily), source:
RelativeBindingSource.Self),
        new Binding(nameof(Entry.FontSize), source:
RelativeBindingSource.Self),
        new Binding(nameof(Entry.FontAttributes), source:
RelativeBindingSource.Self),
    },
    Converter = new StringConcatenationConverter()
});

// in .NET 9
myEntry.SetBinding(Entry.TextProperty, new MultiBinding
{
    Bindings = new Collection<BindingBase>
    {
        Binding.Create(static (Entry entry) => entry.FontFamily,
source: RelativeBindingSource.Self),
        Binding.Create(static (Entry entry) => entry.FontSize,
source: RelativeBindingSource.Self),
        Binding.Create(static (Entry entry) => entry.FontAttributes,
source: RelativeBindingSource.Self),
    },
    Converter = new StringConcatenationConverter()
});
```

> **Important**
>
> Compiled bindings are required instead of string-based bindings in NativeAOT apps, and in apps with full trimming enabled.

Compiled bindings in XAML

In .NET MAUI 8, compiled bindings are disabled for any XAML binding expressions that define the `Source` property, and are unsupported on multi-bindings. These restrictions have been removed in .NET MAUI 9. For information about compiling XAML binding expressions that define the `Source` property, see Compile bindings that define the Source property.

By default, .NET MAUI 9 produces build warnings for bindings that don't use compiled bindings. For more information about XAML compiled bindings warnings, see XAML compiled bindings warnings.

Dependency injection

In a Shell app, you no longer need to register your pages with the dependency injection container unless you want to influence the lifetime of the page relative to the container with the AddSingleton, AddTransient, or AddScoped methods. For more information about these methods, see Dependency lifetime.

Handler disconnection

When implementing a custom control using handlers, every platform handler implementation is required to implement the DisconnectHandler() method, to perform any native view cleanup such as unsubscribing from events. However, prior to .NET MAUI 9, the DisconnectHandler() implementation is intentionally not invoked by .NET MAUI. Instead, you'd have to invoke it yourself when choosing to cleanup a control, such as when navigating backwards in an app.

In .NET MAUI 9, handlers automatically disconnect from their controls when possible, such as when navigating backwards in an app. In some scenarios you might not want this behavior. Therefore, .NET MAUI 9 adds a HandlerProperties.DisconnectPolicy attached property for controlling when handlers are disconnected from their controls. This property requires a HandlerDisconnectPolicy argument, with the enumeration defining the following values:

- `Automatic`, which indicates that handlers will be disconnected automatically. This is the default value of the HandlerProperties.DisconnectPolicy attached property.
- `Manual`, which indicates that handlers will have to be disconnected manually by invoking the DisconnectHandler() implementation.

The following example shows setting the HandlerProperties.DisconnectPolicy attached property:

XAML

```
<controls:Video x:Name="video"
                HandlerProperties.DisconnectPolicy="Manual"
                Source="video.mp4"
                AutoPlay="False" />
```

The equivalent C# code is:

C#

```
Video video = new Video
{
    Source = "video.mp4",
    AutoPlay = false
};
HandlerProperties.SetDisconnectPolicy(video,
HandlerDisconnectPolicy.Manual);
```

In addition, there's a DisconnectHandlers extension method that disconnects handlers from a given IView:

C#

```
video.DisconnectHandlers();
```

When disconnecting, the DisconnectHandlers method will propagate down the control tree until it completes or arrives at a control that has set a manual policy.

Multi-window support

.NET MAUI 9 adds the ability to bring a specific window to the front on Mac Catalyst and Windows with the `Application.Current.ActivateWindow` method:

C#

```
Application.Current?.ActivateWindow(windowToActivate);
```

Native AOT deployment

In .NET MAUI 9 you can opt into Native AOT deployment on iOS and Mac Catalyst. Native AOT deployment produces a .NET MAUI app that's been ahead-of-time (AOT) compiled to native code. This produces the following benefits:

- Reduced app package size, typically up to 2.5x smaller.
- Faster startup time, typically up to 2x faster.
- Faster build time.

For more information, see Native AOT deployment on iOS and Mac Catalyst.

Native embedding

.NET MAUI 9 includes full APIs for native embedding scenarios, which previously had to be manually added to your project:

C#

```
var mauiApp = MauiProgram.CreateMauiApp();

#if ANDROID
var mauiContext = new MauiContext(mauiApp.Services, window);
#else
var mauiContext = new MauiContext(mauiApp.Services);
#endif

var mauiView = new MyMauiContent();
var nativeView = mauiView.ToPlatform(mauiContext);
```

Alternatively, you can use the `ToPlatformEmbedded` method, passing in the `Window` for the platform on which the app is running:

C#

```
var mauiApp = MauiProgram.CreateMauiApp();
var mauiView = new MyMauiContent();
var nativeView = mauiView.ToPlatformEmbedded(mauiApp, window);
```

In both examples, `nativeView` is a platform-specific version of `mauiView`.

To bootstrap a native embedded app in .NET MAUI 9, call the `UseMauiEmbeddedApp` extension method on your `MauiAppBuilder` object:

C#

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();

        builder
            .UseMauiEmbeddedApp<App>();

        return builder.Build();
    }
}
```

For more information, see Native embedding.

Project templates

.NET MAUI 9 adds a **.NET MAUI Blazor Hybrid and Web App** project template to Visual Studio that creates a solution with a .NET MAUI Blazor Hybrid app with a Blazor Web app, which share common code in a Razor class library project.

The template can also be used from `dotnew new`:

.NET CLI

```
dotnet new maui-blazor-web -n AllTheTargets
```

Resource dictionaries

In .NET MAUI 9, a stand-alone XAML ResourceDictionary (which isn't backed by a code-behind file) defaults to having its XAML compiled. To opt out of this behavior, specify `<?xaml-comp compile="false" ?>` after the XML header.

Trimming

Full trimming is now supported by setting the `$(TrimMode)` MSBuild property to `full`. For more information, see Trim a .NET MAUI app.

Trimming incompatibilities

The following .NET MAUI features are incompatible with full trimming and will be removed by the trimmer:

- Binding expressions where that binding path is set to a string. Instead, use compiled bindings. For more information, see Compiled bindings.
- Implicit conversion operators, when assigning a value of an incompatible type to a property in XAML, or when two properties of different types use a data binding. Instead, you should define a TypeConverter for your type and attach it to the type using the TypeConverterAttribute. For more information, see Define a TypeConverter to replace an implicit conversion operator.
- Loading XAML at runtime with the LoadFromXaml extension method. This XAML can be made trim safe by annotating all types that could be loaded at runtime with the DynamicallyAccessedMembers attribute or the DynamicDependency attribute. However, this is very error prone and isn't recommended.
- Receiving navigation data using the QueryPropertyAttribute. Instead, you should implement the IQueryAttributable interface on types that need to accept query parameters. For more information, see Process navigation data using a single method.
- The `SearchHandler.DisplayMemberName` property. Instead, you should provide an ItemTemplate to define the appearance of SearchHandler results. For more information, see Define search results item appearance.

Trimming feature switches

.NET MAUI has trimmer directives, known as feature switches, that make it possible to preserve the code for features that aren't trim safe. These trimmer directives can be used when the `$(TrimMode)` build property is set to `full`, as well as for NativeAOT:

Expand table

MSBuild property	Description
`MauiEnableVisualAssemblyScanning`	When set to `true`, .NET MAUI will scan assemblies for types implementing `IVisual` and for `[assembly:Visual(...)]` attributes, and will register these types. By default, this build property is set to `false`.

MSBuild property	Description
`MauiShellSearchResultsRendererDisplayMemberNameSupported`	When set to `false`, the value of `SearchHandler.DisplayMemberName` will be ignored. Instead, you should provide an ItemTemplate to define the appearance of SearchHandler results. By default, this build property is set to `true`.
`MauiQueryPropertyAttributeSupport`	When set to `false`, `[QueryProperty(...)]` attributes won't be used to set property values when navigating. Instead, you should implement the IQueryAttributable interface to accept query parameters. By default, this build property is set to `true`.
`MauiImplicitCastOperatorsUsageViaReflectionSupport`	When set to `false`, .NET MAUI won't look for implicit conversion operators when converting values from one type to another. This can affect bindings between properties with different types, and setting a property value of a bindable object with a value of a different type. Instead, you should define a TypeConverter for your type and attach it to the type using the TypeConverterAttribute attribute. By default, this build property is set to `true`.
`_MauiBindingInterceptorsSupport`	When set to `false`, .NET MAUI won't intercept any calls to the `SetBinding` methods and won't try to compile them. By default, this build property is set to `true`.
`MauiEnableXamlCBindingWithSourceCompilation`	When set to `true`, .NET MAUI will compile all bindings, including those where the `Source` property is used. If you enable this feature ensure that all bindings have the correct `x:DataType` so that they compile, or clear the data type with `x:Data={x:Null}` if the binding shouldn't be compiled. By default, this build

MSBuild property	Description
	property is only set to `true` when full trimming or Native AOT deployment is enabled.

These MSBuild properties also have equivalent AppContext switches:

- The `MauiEnableVisualAssemblyScanning` MSBuild property has an equivalent AppContext switch named `Microsoft.Maui.RuntimeFeature.IsIVisualAssemblyScanningEnabled`.
- The `MauiShellSearchResultsRendererDisplayMemberNameSupported` MSBuild property has an equivalent AppContext switch named `Microsoft.Maui.RuntimeFeature.IsShellSearchResultsRendererDisplayMemberNameSupported`.
- The `MauiQueryPropertyAttributeSupport` MSBuild property has an equivalent AppContext switch named `Microsoft.Maui.RuntimeFeature.IsQueryPropertyAttributeSupported`.
- The `MauiImplicitCastOperatorsUsageViaReflectionSupport` MSBuild property has an equivalent AppContext switch named `Microsoft.Maui.RuntimeFeature.IsImplicitCastOperatorsUsageViaReflectionSupported`.
- The `_MauiBindingInterceptorsSupport` MSBuild property has an equivalent AppContext switch named `Microsoft.Maui.RuntimeFeature.AreBindingInterceptorsSupported`.
- The `MauiEnableXamlCBindingWithSourceCompilation` MSBuild property has an equivalent AppContext switch named `Microsoft.Maui.RuntimeFeature.MauiEnableXamlCBindingWithSourceCompilationEnabled`.

The easiest way to consume a feature switch is by putting the corresponding MSBuild property into your app's project file (*.csproj), which causes the related code to be trimmed from the .NET MAUI assemblies.

Windows app deployment

When debugging and deploying a new .NET MAUI project to Windows, the default behavior in .NET MAUI 9 is to deploy an unpackaged app. For more information, see Deploy and debug your .NET MAUI app on Windows.

XAML compiler error codes

In .NET MAUI 9, the XAML compiler error codes have changed their prefix from `XFC` to `XC`. Ensure that you update the `$(WarningsAsErrors)`, `$(WarningsNotAsErrors)`, and `$(NoWarn)` build properties in your app's project files, if used, to reference the new prefix.

XAML markup extensions

All classes that implement IMarkupExtension, IMarkupExtension<T>, IValueProvider, and IExtendedTypeConverter need to be annotated with either the RequireServiceAttribute or AcceptEmptyServiceProviderAttribute. This is required due to a XAML compiler optimization introduced in .NET MAUI 9 that enables the generation of more efficient code, which helps reduce the app size and improve runtime performance.

For information about annotating markup extensions with these attributes, see Service providers.

Xcode sync

.NET MAUI 9 includes Xcode sync (`xcsync`), which is a tool that enables you to use Xcode for managing Apple specific files with .NET projects, including asset catalogs, plist files, storyboards, and xib files. The tool has two main commands to generate a temporary Xcode project from a .NET project, and to synchronize changes from the Xcode files back to your .NET project.

You use `dotnet build` with the `xcsync-generate` or `xcsync-sync` commands, to generate or sync these files, and pass in a project file and additional arguments:

.NET CLI

```
dotnet build /t:xcsync-generate
    /p:xcSyncProjectFile=<PROJECT>
    /p:xcSyncXcodeFolder=<TARGET_XCODE_DIRECTORY>
    /p:xcSyncTargetFrameworkMoniker=<FRAMEWORK>
    /p:xcSyncVerbosity=<LEVEL>
```

For more information, see Xcode sync.

Deprecated APIs

.NET MAUI 9 deprecates some APIs, which will be completely removed in a future release.

Frame

The Frame control is marked as obsolete in .NET MAUI 9, and will be completely removed in a future release. The Border control should be used in its place. For more information see Border.

MainPage

Instead of defining the first page of your app using the MainPage property on an Application object, you should set the Page property on a Window to the first page of your app. This is what happens internally in .NET MAUI when you set the MainPage property, so there's no behavior change introduced by the MainPage property being marked as obsolete.

The following example shows setting the Page property on a Window, via the `CreateWindow` override:

C#

```
public partial class App : Application
{
    public App()
    {
        InitializeComponent();
    }

    protected override Window CreateWindow(IActivationState? activa-
tionState)
    {
        return new Window(new AppShell());
    }
}
```

Code that accesses the `Application.Current.MainPage` property should now access the `Application.Current.Windows[0].Page` property for apps with a single window. For apps with multiple windows, use the `Application.Current.Windows` collection to identify the correct window and then access the `Page` property. In addition, each element features a `Window` property, that's accessible when the element is part of the current window, from which the `Page` property can be accessed (`Window.Page`). Platform code can retrieve the app's IWindow object with the `Microsoft.Maui.Platform.GetWindow` extension method.

While the MainPage property is retained in .NET MAUI 9 it will be completely removed in a future release.

Compatibility layouts

The compatibility layout classes in the Microsoft.Maui.Controls.Compatibility namespace have been obsoleted.

Legacy measure calls

The following VisualElement measure methods have been obsoleted:

- VisualElement.OnMeasure
- VisualElement.Measure(Double, Double, MeasureFlags)

These are legacy measure methods that don't function correctly with .NET MAUI layout expectations.

As a replacement, the VisualElement.Measure(Double, Double) method has been introduced. This method returns the minimum size that an element needs in order to be displayed on a device. Margins are excluded from the measurement, but are returned with the size. This is the preferred method to call when measuring a view.

In addition, the SizeRequest struct is obsoleted. Instead, Size should be used.

Upgrade from .NET 8 to .NET 9

To upgrade your .NET MAUI projects from .NET 8 to .NET 9, first install .NET 9 and the .NET MAUI workload with Visual Studio 17.12+, or with Visual Studio Code and the .NET MAUI extension and .NET and the .NET MAUI workloads, or with the standalone installer and the `dotnet workload install maui` command.

Update project file

To update your .NET MAUI app from .NET 8 to .NET 9 open the app's project file (*.csproj*) and change the Target Framework Monikers (TFMs) from 8 to 9. If you're using a TFM such as `net8.0-ios15.2` be sure to match the platform version or remove it entirely. The following example shows the TFMs for a .NET 8 project:

```
XML
```

```
<TargetFrameworks>net8.0-android;net8.0-ios;net8.0-
maccatalyst;net8.0-tizen</TargetFrameworks>
<TargetFrameworks
Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks)
;net8.0-windows10.0.19041.0</TargetFrameworks>
```

The following example shows the TFMs for a .NET 9 project:

XML

```
<TargetFrameworks>net9.0-android;net9.0-ios;net9.0-
maccatalyst;net9.0-tizen</TargetFrameworks>
<TargetFrameworks
Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks)
;net9.0-windows10.0.19041.0</TargetFrameworks>
```

If your app's project file references a .NET 8 version of the Microsoft.Maui.Controls NuGet package, either directly or through the `$(MauiVersion)` build property, update this to a .NET 9 version. Then, remove the package reference for the Microsoft.Maui.Controls.Compatibility NuGet package, provided that your app doesn't use any types from this package. In addition, update the package reference for the Microsoft.Extensions.Logging.Debug NuGet package to the latest .NET 9 release.

If your app targets iOS or Mac Catalyst, update the `$(SupportedOSPlatformVersion)` build properties for these platforms to 15.0:

XML

```
<SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlat-
formIdentifier('$(TargetFramework)')) ==
'ios'">15.0</SupportedOSPlatformVersion>
<SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlat-
formIdentifier('$(TargetFramework)')) ==
'maccatalyst'">15.0</SupportedOSPlatformVersion>
```

When debugging and deploying a new .NET MAUI project to Windows, the default behavior in .NET 9 is to deploy an unpackaged app. To adopt this behavior, see Convert a packaged .NET MAUI Windows app to unpackaged.

Prior to building your upgraded app for the first time, delete the `bin` and `obj` folders. Any build errors and warnings will guide you towards next steps.

Update XAML compiler error codes

XAML compiler error codes have changed their prefix from `XFC` to `XC`, so update the `$(WarningsAsErrors)`, `$(WarningsNotAsErrors)`, and `$(NoWarn)` build properties in your app's project file, if used, to reference the new prefix.

Address new XAML compiler warnings for compiled bindings

Build warnings will be produced for bindings that don't use compiled bindings, and these will need to be addressed. For more information, see XAML compiled bindings warnings.

Update XAML markup extensions

XAML markup extensions will need to be annotated with either the RequireServiceAttribute or AcceptEmptyServiceProviderAttribute. This is required due to a XAML compiler optimization that enables the generation of more efficient code, which helps reduce the app size and improve runtime performance. For more information, see Service providers.

Address deprecated APIs

.NET MAUI 9 deprecates some APIs, which will be completely removed in a future release. Therefore, address any build warnings about deprecated APIs. For more information, see Deprecated APIs.

Adopt compiled bindings that set the Source property

You can opt into compiling bindings that set the `Source` property, to take advantage of better runtime performance. For more information, see Compile bindings that define the Source property.

Adopt compiled bindings in C#

You can opt into compiling binding expressions that are declared in code, to take advantage of better runtime performance. For more information, see Compiled bindings in code.

Adopt full trimming

You can adopt into using full trimming, to reduce the overall size of your app, by setting the `$(TrimMode)` MSBuild property to `full`. For more information, see Trim a .NET MAUI app.

Adopt NativeAOT deployment on supported platforms

You can opt into Native AOT deployment on iOS and Mac Catalyst. Native AOT deployment produces a .NET MAUI app that's been ahead-of-time (AOT) compiled to native code. For more information, see Native AOT deployment on iOS and Mac Catalyst.

.NET for Android

.NET for Android in .NET 9, which adds support for API 35, includes work to reduce build times, and to improve the trimability of apps to reduce size and improve performance. For more information about .NET for Android in .NET 9, see the following release notes:

- .NET for Android 9 RC2
- .NET for Android 9 RC1
- .NET for Android 9 Preview 7
- .NET for Android 9 Preview 6
- .NET for Android 9 Preview 5
- .NET for Android 9 Preview 4
- .NET for Android 9 Preview 3
- .NET for Android 9 Preview 2
- .NET for Android 9 Preview 1

Asset packs

.NET for Android in .NET 9 introduces the ability to place assets into a separate package, known as an *asset pack*. This enables you to upload games and apps that would normally be larger than the basic package size allowed by Google Play. By putting these assets into a separate package you gain the ability to upload a package which is up to 2Gb in size, rather than the basic package size of 200Mb.

> **Important**
>
> Asset packs can only contain assets. In the case of .NET for Android this means items that have the `AndroidAsset` build action.

.NET MAUI apps define assets via the `MauiAsset` build action. An asset pack can be specified via the `AssetPack` attribute:

XML

```xml
<MauiAsset
    Include="Resources\Raw\**"
    LogicalName="%(RecursiveDir)%(Filename)%(Extension)"
    AssetPack="myassetpack" />
```

> **Note**
>
> The additional metadata will be ignored by other platforms.

If you have specific items you want to place in an asset pack you can use the `Update` attribute to define the `AssetPack` metadata:

XML

```xml
<MauiAsset Update="Resources\Raw\MyLargeAsset.txt" AssetPack="myas-
setpack" />
```

Asset packs can have different delivery options, which control when your assets will install on the device:

- Install time packs are installed at the same time as the app. This pack type can be up to 1Gb in size, but you can only have one of them. This delivery type is specified with `InstallTime` metadata.
- Fast follow packs will install at some point shortly after the app has finished installing. The app will be able to start while this type of pack is being installed so you should check it has finished installing before trying to use the assets. This kind of asset pack can be up to 512Mb in size. This delivery type is specified with `FastFollow` metadata.
- On demand packs will never be downloaded to the device unless the app specifically requests it. The total size of all your asset packs can't exceed 2Gb, and you can have up to 50 separate asset packs. This delivery type is specified with `OnDemand` metadata.

In .NET MAUI apps, the delivery type can be specified with the `DeliveryType` attribute on a `MauiAsset`:

XML

```
<MauiAsset Update="Resources\Raw\myvideo.mp4" AssetPack="myasset-
pack" DeliveryType="FastFollow" />
```

For more information about Android asset packs, see Android asset packs.

Android 15 support

.NET for Android in .NET 9 adds .NET bindings for Android 15 (API 35). To build for these APIs, update the target framework of your project to `net9.0-android`:

XML

```
<TargetFramework>net9.0-android</TargetFramework>
```

> **Note**
>
> You can also specify `net9.0-android35` as a target framework, but the number 35 will probably change in future .NET releases to match newer Android OS releases.

64-bit architectures by default

.NET for Android in .NET 9 no longer builds the following runtime identifiers (RIDs) by default:

- `android-arm`
- `android-x86`

This should improve build times and reduce the size of Android `.apk` files. Note that Google Play supports splitting up app bundles per architecture.

If you need to build for these architectures, you can add them to your project file (*.csproj*):

XML

```
<RuntimeIdentifiers>android-arm;android-arm64;android-x86;android-
x64</RuntimeIdentifiers>
```

Or in a multi-targeted project:

XML

```
<RuntimeIdentifiers Condition="$([MSBuild]::GetTargetPlatformIdenti-
fier('$(TargetFramework)')) == 'android'">android-arm;android-
arm64;android-x86;android-x64</RuntimeIdentifiers>
```

Android marshal methods

Improvements to Android marshal methods in .NET 9 has made the feature work more reliably in applications but is not yet the default. Enabling this feature has resulted in a ~10% improvement in performance in a test app.

Android marshal methods can be enabled in your project file (*.csproj*) via the `$(AndroidEnableMarshalMethods)` property:

XML

```
<PropertyGroup>
    <AndroidEnableMarshalMethods>true</AndroidEnableMarshalMethods>
</PropertyGroup>
```

For specific details about the feature, see the feature documentation or implementation on GitHub.

Trimming enhancements

In .NET 9, the Android API assemblies (*Mono.Android.dll*, *Java.Interop.dll*) are now fully trim-compatible. To opt into full trimming, set the `$(TrimMode)` property in your project file (*.csproj*):

XML

```
<PropertyGroup>
    <TrimMode>Full</TrimMode>
</PropertyGroup>
```

This also enables trimming analyzers, so that warnings are introduced for any problematic C# code.

For more information, see Trimming granularity.

.NET for iOS

.NET 9 on iOS, tvOS, Mac Catalyst, and macOS uses Xcode 16.0 for the following platform versions:

- iOS: 18.0
- tvOS: 18.0
- Mac Catalyst: 18.0
- macOS: 15.0

For more information about .NET 9 on iOS, tvOS, Mac Catalyst, and macOS, see the following release notes:

- .NET 9.0.1xx RC2
- .NET 9.0.1xx RC1
- .NET 9.0.1xx Preview 7
- .NET 9.0.1xx Preview 6
- .NET 9.0.1xx Preview 5
- .NET 9.0.1xx Preview 4
- .NET 9.0.1xx Preview 3
- .NET 9.0.1xx Preview 2
- .NET 9.0.1xx Preview 1

Bindings

.NET for iOS 9 introduces the ability to multi-target versions of .NET for iOS bindings. For example, a library project may need to build for two distinct iOS versions:

XML

```xml
<TargetFrameworks>net9.0-ios17.0;net9.0-ios17.2</TargetFrameworks>
```

This will produce two libraries, one using iOS 17.0 bindings, and one using iOS 17.2 bindings.

> **Important**
>
> An app project should always target the latest iOS SDK.

Trimming enhancements

In .NET 9, the iOS and Mac Catalyst assemblies (*Microsoft.iOS.dll*, *Microsoft.MacCatalyst.dll* etc.) are now fully trim-compatible. To opt into full trimming,

set the `$(TrimMode)` property in your project file (*.csproj*):

```xml
<PropertyGroup>
    <TrimMode>Full</TrimMode>
</PropertyGroup>
```

This also enables trimming analyzers, so that warnings are introduced for any problematic C# code.

For more information, see Trimming granularity.

Native AOT for iOS & Mac Catalyst

In .NET for iOS 9, native Ahead of Time (AOT) compilation for iOS and Mac Catalyst takes advantage of full trimming to reduce your app's package size and startup performance. NativeAOT builds upon full trimming, by also opting into a new runtime.

> **Important**
>
> Your app and it's dependencies must be fully trimmable in order to utilize this feature.

NativeAOT requires applications to be built with zero trimmer warnings, in order to prove the application will work correctly at runtime.

See also

- What's new in .NET 9.
- Our Vision for .NET 9